

Supplementary Material for Next: Enabling Continuous UTXO Payments with Direct Guarantees

Lu Hengrun

S1. DETAILS OF THE ORIGINAL-EXECUTION CONNECTION

Earliest conflicting polka. Fix a height and a commit certificate for nonempty value v in round r . Select two honest signers K of that certificate. Each locks v before issuing its precommit. The certificate can be assembled after these signers have advanced; its observation time is irrelevant.

Suppose a higher-round polka for a different value exists, counting nil if it permits unlocking. Select the smallest round $s > r$ with such a polka and, within s , the earliest event τ at which its distinct prevotes exist. Its three voters include a member $m \in K$. Because honest rounds increase, m 's round- s prevote follows its round- r precommit. Without an unlocking justification, m must prevote its locked value. Relocking on v in intervening rounds preserves this restriction. A conflicting justification from a round below s contradicts the choice of s ; one from round s must have existed before m cast the vote counted at τ , contradicting the choice of τ . Evidence from above s cannot precede a subsequent round- s vote. Thus no such polka exists. A higher-round conflicting commit would require one. Same-round conflicting commits require an honest double precommit. This argument depends on the stated lock-transition and round-monotonicity rules, not timely receipt of commit evidence.

Pairing, validation, and execution. Induct on candidate updates: local construction pairs a body with its original parts; remote completion reconstructs the body from checked parts. Moving Locked or Valid transfers both; switching targets clears the old body. Historical installation does not mutate these active candidates. A nonempty precommit follows a matching polka. An honest prevote is justified by actual proposal validation or an earlier lock; tracing the latter decreases the lock round and terminates at validation. The final full-identity guard fixes the original transactions, height, time, and other application context before Finalize. Synchronization and replay supply the same original material. Hence same-height retries cannot introduce a second original body through the stable-hash cache within this call boundary.

Meaning of the two equivalences. Batch refinement compares economic state after a whole batch with the state after its specified sequence of single-item steps. It projects away command identities, revision counters, and materialization tasks. Replay equality instead fixes the entire original command sequence and its block contexts; deterministic execution then reproduces each application hash. These are different relations. Neither compares histories that order parent arrivals and compensations differently. The delay-neutrality theorem of the main text makes that comparison under its own hypotheses, equating final CAL holders and reserve balances rather than application hashes or revision metadata.

S2. SUPPLEMENTARY EXPERIMENTAL METHODS

Build lineage and deployment. Current source-recovery code is committed as d99ec92. The artifact records per-binary SHA-256 fingerprints and configuration for each run. The current nine-service continuation, recovery, storage, and short throughput series use this source-recovery rule. The initial fourteen-service paired runs use the same single-obligation recovery rule before the batch-result tag was separated from the direct-result tag; the current cross-type codec regressions cover that separation. Members in the current storage, in-memory continuation, and short throughput controls run with GOGC=200 and GOMAXPROCS=16; this override is not applied to committee, gateway, or wallet processes.

Archived operating-range runs record base e3870e1 and implementation a083ff6; original continuation records f0c0a274; capital records base 8e1fb40 and implementation 5807546; organization paths record base bb65af69 and verified source 13be856.

The archived batch integration uses `f82f8cd`; structural repair counts were repeated on the current code. The source-failure checkpoint `c1ebf2d` uses the superseded late-source rule and does not support the current recovery claims.

Mac trials share one M4 Max host with 16 CPU cores and 64 GiB RAM; structural microbenchmarks run on Windows amd64. Current memory chains and short throughput trials use in-memory service state and NoSync wallets. Recovery trials use disk-backed committee application state and BlockStore with in-memory members and gateway. Storage controls change only member commits between bbolt NoSync and synchronous transactions. Each run starts from fresh genesis.

Six additional fourteen-service disk-backed chain runs use the same source-recovery rule and user-paid FUEL. Their three fast 100-hop runs take 668.960–688.080 ms; the inter-hop confirmation runs take 114.496–118.742 s. They are a separate storage/deployment configuration from the nine-service results in the main paper.

Timing and aggregation. Certified-receipt latency generally runs from the first actual wallet HTTP send through recipient verification and local atomic recording. Capital and source-failure drivers include a small amount of pre-send construction; their comparisons remain within the same driver family. A local NoSync transaction completion is not a synchronous durability receipt. Public and member completion timestamps record observation of authenticated execution, rather than the committee’s internal commit. The sender’s single monotonic clock measures cross-site ACK latency, including the return path. A recipient can start its successor before the preceding sender receives that ACK. Offered targets, actual send rates, and completion rates including drain are retained separately. Captions distinguish transaction quantiles from statistics across runs. Within-run sample variability is not an estimate of cross-run uncertainty.

Externally funded authorization controller. The capital experiments sample member availability every 200 ms. Let ρ be the declared CAL authorization demand per second, counting both parent and child requests in each workload unit, and let u count unique requests whose certification remains unfinished. The controller sets

$$\begin{aligned} b &= 200 + 100u, \\ H_{\text{low}} &= \rho(0.8 \text{ s}) + b, \\ H_{\text{target}} &= 1.25H_{\text{low}}. \end{aligned}$$

Here the watermarks and burst allowance are in CAL. When the minimum observed member availability A_{min} falls below H_{low} , the proposed grant increment is

$$\Delta B = 100 \left\lceil \frac{1.5(H_{\text{target}} - A_{\text{min}})}{100} \right\rceil,$$

subject to the configured authorization cap and available external funding. At most one increment is in flight. Its public command debits the external source and credits the reserve and grant atomically; following members add the resulting share difference without clearing existing approvals. The experiment supplies ρ in advance rather than estimating unknown production demand. Controller decisions are requests for actual funded transfers, not a mechanism for creating backing capital.

Capital workload accounting. Each condition has one 300-s run with one Worker per member and sufficient user FUEL. Starting from 3,600 CAL, the constant, rate-step, and delayed-parent conditions end at 14,400, 21,600, and 23,300 CAL after external transfers of 10,800, 18,000, and 19,700 CAL. Each completes 6,000 two-hop units, or 12,000 payments, spending 1,128,000 user FUEL. Corresponding fixed-high controls complete; the fixed-low condition closes 46 of 6,000 units, leaves 5,826 unstarted, and records 64 partially approved payments. These categories have different units and are not summed as one payment count. The per-run tables preserve the full seven-condition comparison without implying three repetitions per condition.

Identities, resource use, and retained history. The multi-identity configurations use shared clients; 2,048 wallet identities do not represent 2,048 separate operating system processes. Comparing independent inputs with ten-hop payments, the three-run medians are 12.67 versus 15.24 core-ms per payment and 59.32 versus 71.83 KiB of HTTP payload per payment. Memory reporting separates total node RSS and retained history from active obligations, pending work, and outboxes. An increasing historical footprint is therefore not reported as an increasing payment backlog. Per-run resource summaries and their archived source locations are included in the artifact index.

Endpoint-specific Lightning reference. The archived deployment uses LND v0.21.3-beta and Bitcoin Core 31.1 in regtest, with synchronous LND persistence and its 50-ms channel-commit setting. Each of three fresh deployments

TABLE S1

APPEND-ONLY COMPENSATION RECORDS AND AUTHORIZED REPRESENTATION UPDATES (C3). BOTH CLOSE THE ACCOUNT THROUGH THE SAME ECONOMIC TRANSITION; THEY DIFFER IN THE STORED HISTORICAL REPRESENTATION.

	Append-only record	Representation update (C3)
Economic closure	Reserve debit and obligation state in the public command; successors unchanged.	Same transition and atomic step, committed with a revision task.
Historical representation and original commitments	Original body unchanged; the funding source of a consumed output is found through an index of compensation records.	Stored body names the reserve debit; transaction identity, owner authorization, and complete block identity (H_b, P_b) are unchanged and verified.
Actual readers	Any reader of the command history and records.	Repair construction, installation, and local inspection read the revised body; wallets, synchronization, and replay read original commands.
Storage and replay	Original blocks plus records; replay runs original commands and compensation events.	Original blocks plus revised bodies, revisions, and tasks; replay runs original commands and repair events; same-block repairs share part adaptation and installation advances to the latest committed revision.
Progress dependency	Public execution of the compensation record.	Compensation commits with its validated revision and requires three adaptation contributions.
Status	Design baseline; not implemented as a mode and not measured.	Implemented; costs in the main paper.

has two nodes and a direct channel. After 20 warm-up transfers, each run issues 100 serial, alternating 10,000-satoshi payments using prepared invoices, with receiver `SETTLED` as the completion event; the reported P50/P95 are 282.12/320.17 ms. A separate Next run sends 300 payments at 100 payments/s for 3 s using in-memory service state and NoSync wallet storage. Recipient-certified receipt has P50/P95 of 0.9625/1.2075 ms. Workload concurrency, durability, and completion semantics differ, so these records are deployment references rather than a normalized acceleration ratio.

A. Archived Operating-Range Series

The archived operating-range series complements the continuation results. One active organization uses sponsored fees, reused addresses, and independent final inputs. Five runs of 150,000 payments each all finish (Fig. S1). At a target of 2,000 payments/s, two runs achieve completion rates of 1,948.66–1,950.45 payments/s, including drain, with receipt P50s of 2.521–2.566 ms. Raising the target to 2,200 yields 2,103.63–2,106.29 completed payments/s. At 2,400, the achieved rate is 2,174.68, P50/P95 rise to 7.174/89.38 ms, and the 4,096-entry pending admission limit records 1,635 hits. The operating curve pairs achieved completion rates with their corresponding queue pressure.

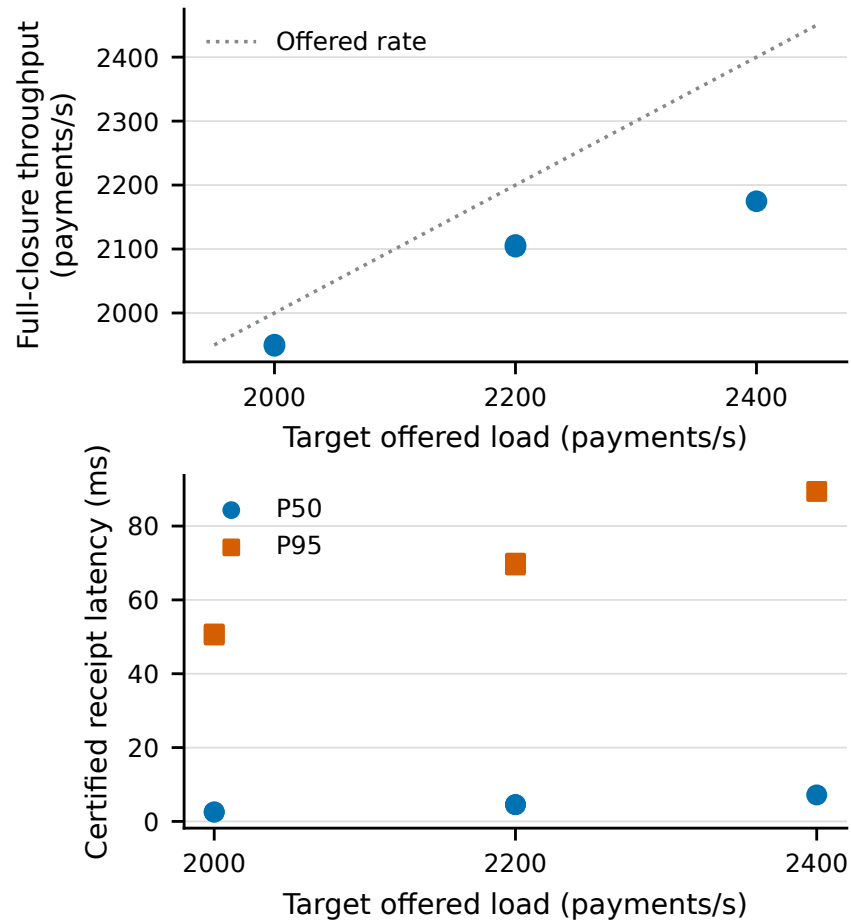


Fig. S1. Operating points versus target offered load. Top: completion throughput (payments/s), including drain. Bottom: receipt latency (ms). Each point is one 150,000-payment run: two at target 2,000, two at 2,200, and one at 2,400. P50/P95 summarize transactions within each run, not cross-run uncertainty. All services share one host.

S3. PER-RUN EXPERIMENTAL RECORDS

TABLE S2

ARCHIVED ONE-ORGANIZATION OPERATING-POINT RUNS. EACH RUN SENDS AND COMPLETES 150,000 PAYMENTS. INJECTION AND COMPLETION ARE IN PAYMENTS/S; RECEIPT QUANTILES ARE IN MS. COMPLETION INCLUDES FINAL BACKGROUND DRAIN. PEAK IS THE MAXIMUM NUMBER OF PAYMENTS WITH UNFINISHED BACKGROUND PROCESSING; LIMIT HITS COUNTS TRIGGERS OF THE 4,096-UNFINISHED-PAYMENT LIMIT.

Run	Target	Injection	Completion	P50	P95	Peak	Limit hits
1	2000	1,964.53	1,948.66	2.521	50.499	2159	0
2	2000	1,969.65	1,950.45	2.566	50.761	2205	0
3	2200	2,127.70	2,103.63	4.538	69.463	2655	0
4	2200	2,127.15	2,106.29	4.553	69.945	2548	0
5	2400	2,206.95	2,174.68	7.174	89.376	4096	1635

TABLE S3

CURRENT SOURCE-RECOVERY RUNS. EACH OFFERS 1,200 TWO-HOP UNITS (2,400 PAYMENTS) OVER 60 SECONDS. RECEIPT QUANTILES FOR NORMAL CHILD PAYMENTS ARE IN MS; ELAPSED AND DEADLINE-TO-ALL-REPLICA-INSTALLATION OBSERVATION MEDIAN ARE IN SECONDS. THE LATTER INCLUDES 250-MS POLLING. EVERY RUN ENDS WITH A 60,000-CAL RESERVE, ZERO NET EXPENDITURE, AND ZERO GAP.

Seed	Withheld	Paid/recovered	Elapsed	Child P50	Child P95	Install P50
23	0%	0/0	61.540	1.251	1.620	–
23	1%	12/12	90.225	1.185	1.432	1.759
23	5%	60/60	90.096	1.191	1.536	1.257
37	0%	0/0	61.595	1.199	1.559	–
37	1%	12/12	91.935	1.190	1.479	1.509
37	5%	60/60	91.530	1.192	1.426	1.257
59	0%	0/0	61.550	1.191	1.461	–
59	1%	12/12	90.595	1.193	1.452	1.761
59	5%	60/60	90.346	1.194	1.494	1.507

TABLE S4

ARCHIVED FINITE-RESERVE RUNS. ONE 300-S RUN PER CONDITION; CAPITAL VALUES ARE CAL. CLOSED COUNTS REFER TO TWO-HOP UNITS. ADAPTIVE INCREASES ARE ACTUAL TRANSFERS FROM AN EXTERNAL FUNDING ACCOUNT. MEAN IS THE TIME-AVERAGE ORGANIZATION CAL BALANCE OBTAINED BY INTEGRATING BALANCE SAMPLES OVER THE 300-S INPUT WINDOW.

Condition	Initial	Added	Final	Mean	Closed
Constant / fixed low	3,600	0	3,600	3,600.0	46/6000
Constant / fixed high	14,400	0	14,400	14,400.0	6000/6000
Constant / adaptive	3,600	10,800	14,400	13,801.8	6000/6000
Step / fixed high	21,600	0	21,600	21,600.0	6000/6000
Step / adaptive	3,600	18,000	21,600	14,078.7	6000/6000
Delayed / fixed high	28,800	0	28,800	28,800.0	6000/6000
Delayed / adaptive	3,600	19,700	23,300	20,611.2	6000/6000

TABLE S5

SYNCHRONOUS-MEMBER CONTINUATION CONTROLS: 100 HOPS PER RUN, NINE DISK-BACKED SERVICES, USER-PAID FUEL, AND NoSYNC WALLETS. DURATIONS END AT THE LAST CERTIFIED RECEIPT AND ARE IN SECONDS. N/S ORDER APPLIES TO THE TWO FAST MODES; THE SYNCHRONOUS WAITING RUN FOLLOWS EACH PAIR.

Pair	Order	NoSync fast	Sync fast	Sync wait
1	N/S	0.681127	4.492812	64.969480
2	S/N	0.705604	4.309984	65.193522
3	N/S	0.636821	4.470844	65.012947
Median		0.681127	4.470844	65.012947